

CYBER SECURITY TASK 3 (2026) | FUTURE INTERNS

API Security Risk Analysis

Read-only security assessment of the ReqRes demo API (<https://reqres.in>)

Target API	ReqRes (reqres.in) - public demo / testing API
Assessment type	Read-only, documentation and response-based API security risk analysis
Tools used	Postman, ReqRes Request Builder / Response Viewer
Prepared by	[Your Name]
Report date	29 September 2026
Classification	Educational / Public demo target - no production systems tested

1. Executive Summary

APIs power modern SaaS platforms, mobile apps and dashboards, which makes them a prime target for attackers. This report documents a read-only security risk analysis of the ReqRes public demo API. The objective was to think like a security consultant: observe how the API authenticates callers, what data it returns, and where the design would create business risk if it were used in a real product.

Three representative requests were made (a user list, a login and a single-user lookup). All returned HTTP 200. No exploitation, bypass, flooding or brute-force activity was performed. Seven risks were identified from the observed behaviour:

ID	Finding	Severity	OWASP API 2023
F-01	Sequential object IDs and no visible object-level authorization	High	API1: BOLA
F-02	Excessive data exposure (PII in user objects)	Medium	API3: BOPLA
F-03	Shared static API key as the sole authentication mechanism	Medium	API2: Broken Auth
F-04	Minimal / weak token design in the login response	Medium	API2: Broken Auth
F-05	No rate limiting evidenced on any endpoint	Medium	API4: Resource Consumption
F-06	Unnecessary metadata and third-party links in API responses	Low	API8: Misconfiguration
F-07	Client-controlled query parameters, validation not evidenced	Low	API8: Misconfiguration

Overall risk rating: MEDIUM-HIGH if this design were used in production. Because ReqRes is a demo service that intentionally returns fake data and states that it is a read-only demo endpoint, the ratings below describe the risk of the observed patterns, not a claim that ReqRes itself is defective.

The most important recommendations are to enforce server-side object-level authorization, return only the fields a client genuinely needs, replace shared static keys with scoped per-client credentials, and add rate limiting with clear 429 responses.

2. Scope and Ethics

2.1 In scope

- ReqRes public demo API at <https://reqres.in> (a service designed for testing and learning).
- Read-only requests: GET /api/users?page=2, GET /api/users/2, and the safe demo POST /api/login preset.
- Inspection of request headers, authentication behaviour, status codes, latency, response size and response body.

2.2 Out of scope / not performed

- Exploitation, authentication bypass, or attempts to access data beyond what the demo presets return.
- Flooding, load testing or denial-of-service testing (so rate limiting could not be actively verified).
- Any testing of private or production APIs.

All activity followed the task rules: public demo target, low request volume, no destructive actions, and no attempt to bypass controls. Findings describe risk, they do not demonstrate exploitation.

3. Methodology

The assessment followed the step-by-step approach defined in the task and was aligned to the OWASP API Security Top 10 (2023) and the API Security Checklist:

1. **Select API.** Chose the ReqRes demo API as a safe, legal target.
2. **Review documentation.** Reviewed the ReqRes documentation and the built-in request presets (List users, Single user, Register, Login, Update, Delete).
3. **Test endpoints.** Obtained a free ReqRes API key through self-service sign-up, ran three representative requests in Postman with the key sent in the x-api-key header, and captured the request and response views as evidence.
4. **Inspect.** Inspected authentication (x-api-key header, login token), request headers, status codes, latency, response size and the full response body.
5. **Identify risks.** Compared observations against OWASP API Top 10 categories: unauthenticated endpoints, excessive data exposure, weak tokens, authorization, rate limiting and input validation.
6. **Classify severity.** Rated each finding Low / Medium / High using the criteria below.
7. **Recommend and document.** Wrote business-language impact statements and concrete remediation steps.

3.1 Severity criteria

Rating	Definition
High	Weakness that could let an attacker read or change other users' data, or gain unauthorized access, with low effort. Fix immediately.
Medium	Weakness that increases the chance or impact of compromise, or that depends on additional conditions to be exploited. Fix within a planned cycle.
Low	Hardening gap or minor information disclosure with limited direct impact. Fix as part of routine improvement.

4. API Overview and Endpoints Tested

ReqRes is a hosted REST API used for front-end and API-client practice. Requests are sent over HTTPS to [https://reqres.in/api/...](https://reqres.in/api/) and responses are JSON. Access uses an x-api-key header; the key was obtained free of charge through self-service sign-up and is injected automatically on every call in the request builder.

#	Method	Endpoint	Status	Latency	Size	Evidence
1	GET	/api/users?page=2	200	504 ms	1.5 KB	Figure 1: user list, page 2
2	POST	/api/login	200	505 ms	436 B	Figure 2: login preset (success)
3	GET	/api/users/2	200	249 ms	777 B	Figure 3: single user, id 2

5. Test Evidence and Observations

5.1 GET /api/users?page=2 (List users)

The list endpoint accepted the page query parameter and returned a paginated response (page 2 of 2, 6 users per page, 12 in total). Each user object contained id, email, first_name, last_name and avatar. The x-api-key header was injected automatically and no user login was needed.

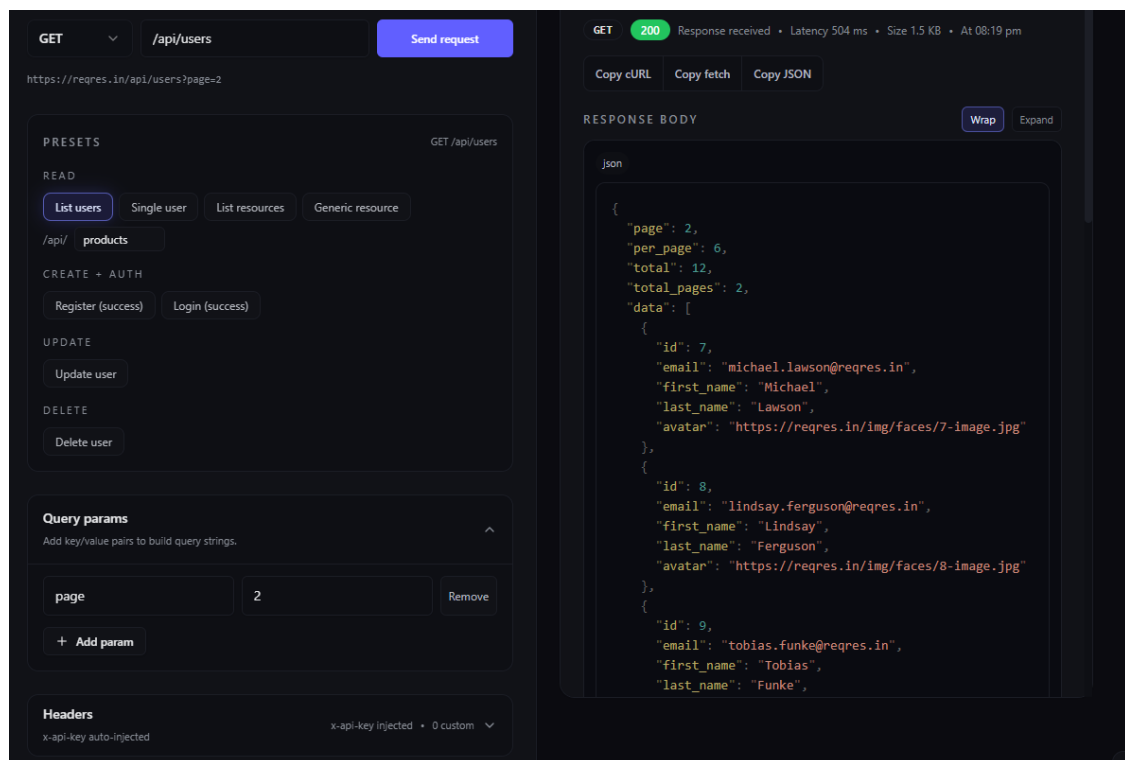


Figure 1: GET /api/users?page=2 returns 200 OK with user emails, names and avatars.

- Sequential numeric IDs are visible (7, 8, 9 ...), which reveals the ID pattern used across the API.
- Email addresses (personal data) are returned in bulk for every user in the list.
- Pagination is present and bounded to 6 records per page, which is a good practice.

5.2 POST /api/login (Login preset)

The login preset returned 200 OK and an access token in the JSON body, together with a _meta object containing documentation links, an upgrade URL, an experiment variant and a marketing message. The message states that this is a read-only demo endpoint.

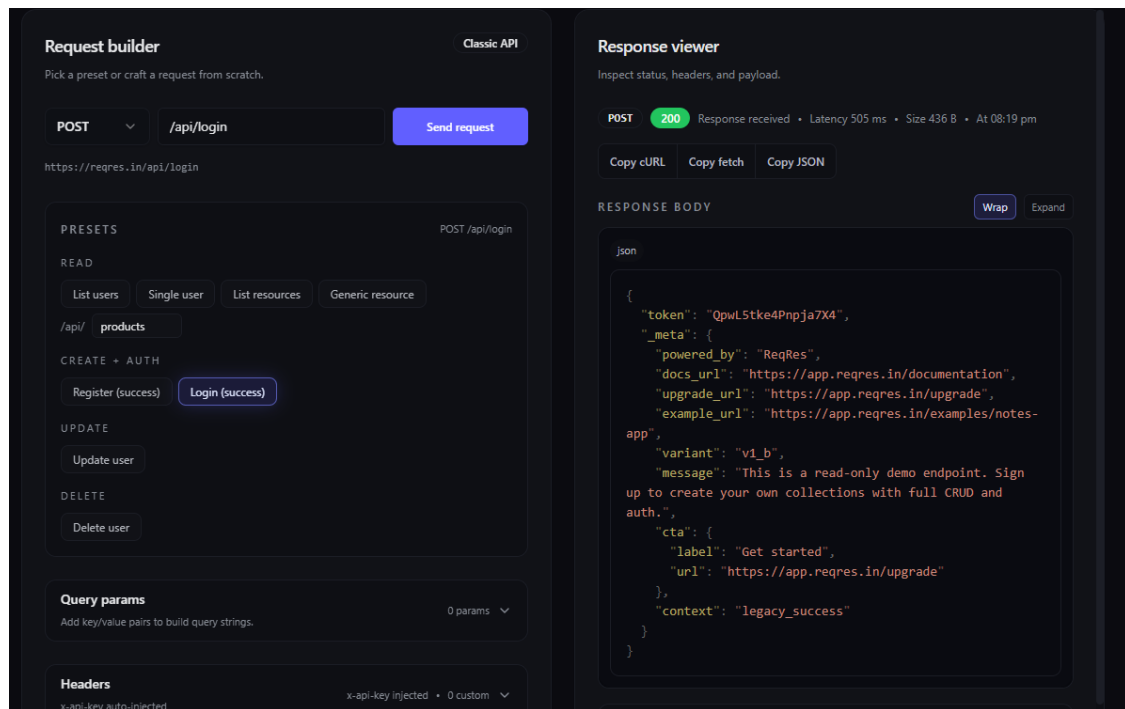


Figure 2: POST `/api/login` returns 200 OK with a token and additional `_meta` / `cta` fields.

- The token (QpwL5tke4Pnpja7X4) is a short 17-character opaque string. No `token_type`, `expires_in`, refresh token or scope is returned, so lifetime and permissions cannot be determined by the client.
- The response body includes marketing and experiment fields (`variant`, `context`, `cta`) that have nothing to do with authentication.
- Credentials used were the demo preset values only. The request body is not shown in the screenshot.

5.3 GET `/api/users/2` (Single user)

Requesting a single user by ID returned the full profile for user 2 in 249 ms. The response also included a support object linking to a third-party website with UTM tracking parameters, plus the same `_meta` block seen on the login response.

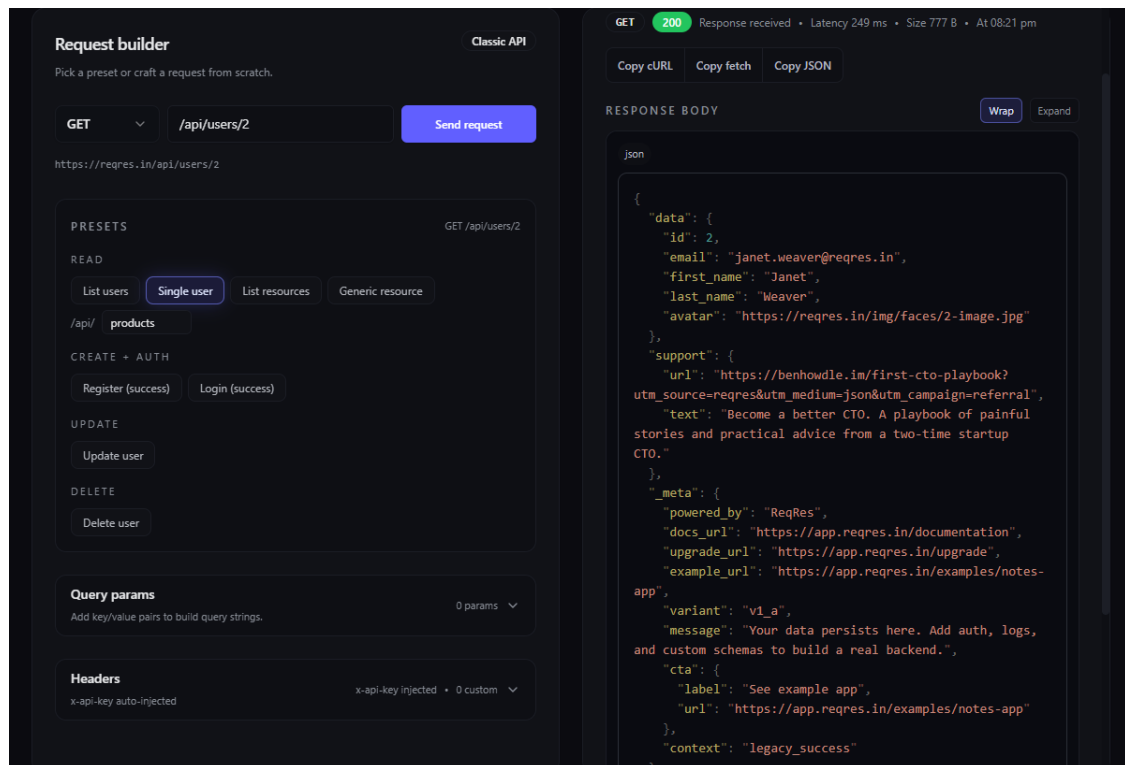


Figure 3: GET /api/users/2 returns 200 OK with the user profile, a third-party support URL and _meta fields.

- The record was returned using only the injected x-api-key; no user-specific token was required or bound to the request.
- The support.url field points to an external domain and carries tracking parameters.
- _meta.variant is v1_a here but v1_b on the login response, revealing internal A/B experiment information.

6. Detailed Findings

F-01 Sequential object IDs and no visible object-level authorization

Severity	High
OWASP API Top 10 (2023)	API1:2023 Broken Object Level Authorization
Affected endpoint(s)	GET /api/users/{id}, GET /api/users
Evidence	Figure 1 shows sequential IDs (7, 8, 9). Figure 3 shows GET /api/users/2 returning a full profile using only the shared x-api-key.
Description	User records are addressed by small, predictable integers. Access appears to depend only on possession of the shared API key, not on any relationship between the caller and the record being requested. This is the classic pattern behind BOLA / IDOR issues. Only a single read was made; no enumeration was attempted, in line with the ethics rules.

Business impact	If the key leaks, or in a real product where users should only see their own data, an attacker could walk through IDs and harvest every customer profile. That means privacy breaches, regulatory exposure (for example GDPR or India's DPDP Act 2023), notification costs and loss of customer trust.
Remediation	• Enforce object-level authorization on the server for every request: verify the caller is allowed to access that specific record. • Use non-sequential identifiers (UUIDv4 / ULID) for public-facing resources. • Bind access tokens to a user identity and scopes, and check them on each object access. • Log and alert on sequential-ID scanning patterns; add automated authorization tests to CI.

F-02 Excessive data exposure (PII in user objects)

Severity	Medium
OWASP API Top 10 (2023)	API3:2023 Broken Object Property Level Authorization
Affected endpoint(s)	GET /api/users, GET /api/users/{id}
Evidence	Figures 1 and 3: every user object contains email, first_name, last_name and avatar URL. The list returns six such records per call.
Description	The API returns full user profiles including email addresses, whether or not the calling client needs them. Extra non-data fields (support, _meta, cta) are also returned with every response.
Business impact	Exposed emails and names support phishing and social-engineering campaigns and become personal-data liabilities. Over-sharing also makes any future authorization bug more damaging.
Remediation	• Return only the fields the client needs, using an explicit allow-list (DTO / serializer) rather than dumping the whole record. • Mask or omit email addresses in list views unless the caller's role requires them. • Review every response schema against a data-classification list before release.

F-03 Shared static API key as the sole authentication mechanism

Severity	Medium
OWASP API Top 10 (2023)	API2:2023 Broken Authentication
Affected endpoint(s)	All endpoints
Evidence	Figures 1 to 3 show the Headers panel: "x-api-key auto-injected". All three requests succeeded with no user sign-in.
Description	A single key sent in a header authorizes the requests, and the key used here was issued free through self-service sign-up, so possessing a key says little about who the caller is. Static keys cannot identify which user or application made a call, are hard to rotate, and grant the same access to whoever holds them.
Business impact	A leaked key (for example embedded in front-end code, a repository or a log) gives full access until someone notices and rotates it. Incident investigation is difficult because activity cannot be tied to an individual.
Remediation	• Issue separate keys per client with least-privilege scopes, expiry and rotation. • Use OAuth 2.0 / OpenID Connect for user-level access instead of a single shared key. • Keep keys server-side only; store them in a secrets manager and scan repositories for leaks. • Monitor key usage and support instant revocation.

F-04 Minimal / weak token design in the login response

Severity	Medium
OWASP API Top 10 (2023)	API2:2023 Broken Authentication
Affected endpoint(s)	POST /api/login
Evidence	Figure 2: token "QpwL5tke4Pnpja7X4" (17 characters). No token_type, expires_in, refresh_token or scope fields are present.
Description	The token is short by modern standards, its randomness cannot be verified, and the response gives no expiry or scope information. Whether it is revoked on logout or bound to a client is unknown. Only the success path was tested.
Business impact	Long-lived or guessable tokens allow account takeover if intercepted or brute-forced, and clients cannot manage token lifetime safely.

Remediation	• Use short-lived signed tokens (JWT with exp, iss, aud) or high-entropy opaque tokens of at least 128 bits. • Return token_type and expires_in, and use rotating refresh tokens. • Send Cache-Control: no-store on authentication responses and support server-side revocation. • Protect the login route against credential stuffing (rate limits, lockout or CAPTCHA, MFA).
--------------------	---

F-05 No rate limiting evidenced on any endpoint

Severity	Medium
OWASP API Top 10 (2023)	API4:2023 Unrestricted Resource Consumption
Affected endpoint(s)	All endpoints, especially POST /api/login
Evidence	The response viewers in Figures 1 to 3 show status, latency and size but no rate-limit information (such as X-RateLimit-* or Retry-After headers). Active flood testing was out of scope, so this is unverified rather than confirmed.
Description	Without enforced limits, callers can send unlimited requests. The absence of limit indicators is a risk signal that should be checked in the response headers tab and the provider's documentation.
Business impact	Unthrottled APIs enable data scraping, password-guessing on login, denial of service and unexpected infrastructure cost.
Remediation	• Apply per-key, per-IP and per-user rate limits at the API gateway or WAF, with stricter limits on /login. • Return HTTP 429 with a Retry-After header and publish limits in the documentation. • Cap page size (per_page) and enforce quotas; alert on abnormal traffic.

F-06 Unnecessary metadata and third-party links in API responses

Severity	Low
OWASP API Top 10 (2023)	API8:2023 Security Misconfiguration
Affected endpoint(s)	POST /api/login, GET /api/users/{id}
Evidence	Figures 2 and 3: _meta (docs_url, upgrade_url, example_url, variant, context: "legacy_success"), cta, and support.url pointing to a third-party domain with UTM tracking parameters. The variant differs between calls (v1_b and v1_a).
Description	The API mixes marketing, experiment and internal context fields with functional data. This discloses internal experimentation and enlarges the response contract.
Business impact	If a client application renders these URLs, a compromised or malicious link could lead users to phishing or malvertising. Internal labels also help attackers understand the platform's behaviour.
Remediation	- Remove non-functional fields from production API responses and keep marketing content separate. - If external links are required, validate them against an allow-list and never render them blindly. - Keep a consistent, documented response schema and disable debug / context fields.

F-07 Client-controlled query parameters, validation not evidenced

Severity	Low
OWASP API Top 10 (2023)	API8:2023 Security Misconfiguration
Affected endpoint(s)	GET /api/users?page={n}
Evidence	Figure 1: the page parameter (value 2) is accepted and echoed back with per_page, total and total_pages. Unexpected values were deliberately not sent, to stay within the read-only scope.
Description	Query parameters are client-controlled. The assessment could not confirm how the API handles invalid, very large, negative or malformed values, or how it words error messages.
Business impact	Poor validation can lead to errors that leak internals, performance problems from oversized requests, or injection issues on other endpoints.

Remediation

- Validate type, range and length of every parameter against a schema (for example OpenAPI) and reject unknown parameters.
- Enforce a maximum per_page value.
- Return generic, consistent error messages without stack traces or internal details.

7. Positive Observations

- Traffic uses HTTPS, protecting data in transit.
- Access is tied to an issued API key rather than being fully anonymous, which gives the provider a basic way to identify and revoke clients.
- List responses are paginated and bounded (6 records per page).
- Responses are clean, consistent JSON with correct HTTP status codes.
- No passwords or password hashes appear in user objects.
- Response times were low (249 to 505 ms) with small payloads (436 B to 1.5 KB).
- The service clearly identifies its endpoints as read-only demo endpoints.

8. Remediation Roadmap

Priority	Findings	Recommended action
High	F-01, F-02	Immediately: add server-side object-level authorization, adopt non-sequential IDs and trim responses to the minimum required fields.
Medium	F-03, F-04, F-05	Within 30 days: move to scoped per-client credentials or OAuth 2.0, redesign the token format, and add gateway-level rate limiting.
Low	F-06, F-07	Next release cycle: remove non-functional metadata, add schema validation and consistent error handling.

9. Limitations

- Requests sent without the API key were not tested, so whether any endpoint is fully open could not be confirmed.
- Only three requests against a public demo API were analysed; results do not represent a full penetration test.
- Rate limiting and input validation could not be actively tested because flooding and malformed-input attacks are out of scope.
- Response headers and the login request body were not captured in the screenshots, so header-level controls (for example HSTS, CORS, cache headers) were not assessed.
- ReqRes returns fictional data by design, so severity ratings reflect what the same pattern would mean in a production SaaS product.

10. Conclusion

The ReqRes demo API behaves as expected for a testing service, but it illustrates several patterns that are common in real SaaS APIs and that regularly lead to breaches: predictable object IDs, over-sharing of personal data, reliance on a shared static key, thin token design and missing throttling signals. Addressing the High and Medium findings first would significantly reduce the likelihood and impact of unauthorized data access. This exercise demonstrates a repeatable, ethical approach to API security review that can be applied to client engagements.

References

- OWASP API Security Top 10 (2023): <https://github.com/OWASP/API-Security>
- API Security Checklist: <https://github.com/shieldfy/API-Security-Checklist>
- ReqRes API testing platform: <https://reqres.in>
- Public APIs collection: <https://github.com/public-apis/public-apis>